

Nazwa kwalifikacji: **Projektowanie, programowanie i testowanie aplikacji**  
Oznaczenie kwalifikacji: **INF.04**  
Numer zadania: **05**

Wypełnia zdający

Numer Identyfikacyjny zdającego\*

|  |  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|--|
|  |  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|--|

Miejsce na numer ośrodka

Czas trwania egzaminu: **180** minut.

INF.04-KG-26.05

## EGZAMIN ZAWODOWY

Rok 2026

CZĘŚĆ PRAKTYCZNA

**PODSTAWA PROGRAMOWA  
2019**

### Instrukcja dla zdającego

1. Na pierwszej stronie arkusza egzaminacyjnego wpisz w oznaczonym miejscu swój numer Identyfikacyjny.
2. Na KARCIE OCENY wpisz swoje dane:
  - swój numer Identyfikacyjny,
  - oznaczenie kwalifikacji,
  - numer zadania,
  - numer stanowiska.
3. Sprawdź, czy arkusz egzaminacyjny zawiera 7 stron i nie zawiera błędów. Ewentualny brak stron lub inne usterki zgłoś przez podniesienie ręki przewodniczącemu zespołu nadzorującego.
4. Zapoznaj się z treścią zadania oraz stanowiskiem egzaminacyjnym. Masz na to 10 minut. Czas ten nie jest wliczany do czasu trwania egzaminu.
5. Czas rozpoczęcia i zakończenia pracy zapisze w widocznym miejscu przewodniczący zespołu nadzorującego.
6. Wykonaj samodzielnie zadanie egzaminacyjne. Przestrzegaj zasad bezpieczeństwa i organizacji pracy.
7. Po zakończeniu wykonania zadania pozostaw arkusz egzaminacyjny z rezultatami oraz KARTĘ OCENY na swoim stanowisku lub w miejscu wskazanym przez przewodniczącego zespołu nadzorującego.
8. Po uzyskaniu zgody zespołu nadzorującego możesz opuścić salę/miejsce przeprowadzania egzaminu.

**Powodzenia!**

\* w przypadku braku numeru ID – seria i numer paszportu lub innego dokumentu potwierdzającego tożsamość

## Zadanie egzaminacyjne

*UWAGA: katalog z rezultatami pracy oraz płytę należy opisać numerem zdającego, którym został podpisany arkusz. Dalej w zadaniu numer ten jest nazwany numerem zdającego.*

Wykonaj aplikację konsolową oraz webową według wskazań. Udokumentuj obie aplikacje zrzutami ekranu. Wykorzystaj konto **Egzamin** bez hasła.

Utwórz folder i nazwij go numerem zdającego. W folderze utwórz podfoldery: *konsola*, *webowa*, *testy*. Po wykonaniu każdej aplikacji, jej pełny kod (cały folder projektu) spakuj do archiwum. Następnie pozostaw w podfolderze jedynie pliki źródłowe, których treść była modyfikowana, plik wykonywalny, jeśli jest to możliwe oraz spakowane archiwum.

### Część I. Aplikacja konsolowa

Za pomocą narzędzi do tworzenia aplikacji konsolowych zaimplementuj funkcję obliczającą dany wyraz ciągu Fibonacciego.

Założenia programu

- Program wykonywany w konsoli.
- Język programowania zgodny z zainstalowanym na stanowisku egzaminacyjnym, jeden z: C++, C#, Java, Python.
- Podejście obiektowe lub strukturalne.
- Program sprawdza dany wyraz ciągu Fibonacciego.
- Program powinien być zapisany czytelnie, z zachowaniem zasad czystego formatowania kodu, należy stosować znaczące nazwy zmiennych i funkcji.

*Ciąg Fibonacciego*

$$F_n = \begin{cases} 0 & \text{dla } n = 0, \\ 1 & \text{dla } n = 1, \\ F_{n-1} + F_{n-2} & \text{dla } n > 1. \end{cases}$$

Źródło: [https://pl.wikipedia.org/wiki/Ci%C4%85g\\_Fibonacciego](https://pl.wikipedia.org/wiki/Ci%C4%85g_Fibonacciego) ; dostęp: 04.02.2026

Definicja 1 – Ciąg Fibonacciego

Założenia dotyczące kodu

- Program zawiera metodę/funkcję obliczającą n-ty wyraz ciągu.
- Metoda obliczająca posiada:
  - argument liczbę całkowitą – n-ty wyraz ciągu
  - implementację rekurencyjnie ciągu Fibonacciego zgodnie z definicją 1.
  - zwraca wynik jako długą liczbę całkowitą
  - gdy użytkownik poda liczbę mniejszą od 0, to program wyrzuca wyjątek argumentu o treści: "Liczba n musi być większa lub równa 0"

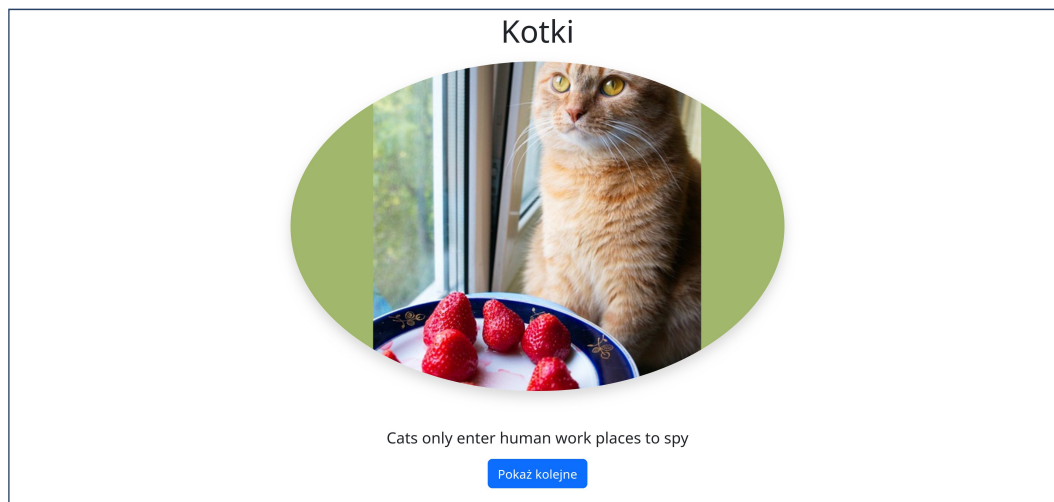
Sprawdź działanie metody lub funkcji sprawdzającej. Program powinien nawiązać czytelną komunikację z użytkownikiem. Do kodu należy dołączyć testy jednostkowe opisane w części III zadania egzaminacyjnego.

Po uruchomieniu programu wykonaj zrzuty ekranu dokumentujące interakcje programu dla różnych wartości roku. Zrzuty ekranu powinny obejmować cały obszar ekranu, z widocznym paskiem zadań oraz środowiskiem programistycznym. Liczba zrzutów ekranu powinna odpowiadać wszystkim możliwym interakcjom użytkownika z programem. Zrzuty ekranu zapisz w folderze konsolowa pod nazwami *konsola1.png*, *konsola2.png*, itd.

Kod aplikacji przygotuj do nagrania na płytę. W folderze konsolowa powinno znaleźć się archiwum całego projektu o nazwie *konsola.zip*, skopiowany z projektu plik z kodem źródłowym programu oraz plik wykonywalny, jeżeli istnieje.

## Część II. Aplikacja webowa

Za pomocą środowiska programistycznego dostępnego na stanowisku egzaminacyjnym, wykonaj aplikację webową będącą graficznym interfejsem do wyświetlania losowego zdjęcia kota i faktu na ich temat. Do zbudowania aplikacji wykorzystaj grafiki i bazę danych z archiwum **pliki5.7z** zabezpieczonego hasłem: **Klakier92^**



Obraz 1. Aplikacja webowa, w stanie początkowym

Na obrazie 1 przedstawiono ideę aplikacji. W zależności od zastosowanego środowiska programistycznego wygląd może nieznacznie się różnić.

### Opis wyglądu aplikacji

- Okno zawiera kontrolki rozmieszczone zgodnie z obrazem 1. Są to:
  - Wyśrodkowany nagłówek 1 stopnia o treści: *"Kotki"*
  - Wyśrodkowany, zaokrąglony w elipsę i wycieniowany obraz kota o tekście alternatywnym: *"kot"*
  - Wyśrodkowany nagłówek 4 stopnia w którym znajduje się fakt
  - Wyśrodkowany przycisk o treści *"Pokaż kolejne"*

### Opis działania aplikacji

- Po uruchomieniu aplikacji domyślnie jest wyświetlone losowe zdjęcie kota i fakt
- Po naciśnięciu przycisku *"Pokaż kolejne"*, skrypt wykonuje zapytanie o nowe zdjęcie kota i fakt

| fakty |              |    |
|-------|--------------|----|
| id    | INT(11)      | PK |
| fakt  | VARCHAR(250) | NN |

Obraz 2. Aplikacja webowa, na stronie kalkulatora (wynik)

#### Założenia API:

- Działa na porcie ustalonym w pliku zmiennych środowiskowych, inaczej 3000
- Dane do połączenia z bazą są przechowywane w pliku zmiennych środowiskowych
- Baza danych kotki pochodzi z archiwum dołączonego do arkusza\*.
- Posiada 3 ścieżki:
  - ścieżkę `/fact` (GET) – zwracającą JSON-a z losowym faktem z bazy
  - ścieżkę `/cat` (GET) – zwracającą losowe zdjęcie kota
  - ścieżkę `/cat/<NUMER ID>` (GET) – zwracającą zdjęcie kota po jego numerze ID wg. kolejności w katalogu plików
- ścieżka zwracająca zdjęcie kota po jego numerze sprawdza czy identyfikator nie jest poza zakresem liczby plików. Górna granica nie może być ustawiona statycznie, ponieważ jest to uwarunkowane projektem późniejszej rozbudowy galerii. W przypadku wyjścia poza granicę zwraca komunikat błędu z odpowiednim kodem statusu HTTP.
- Każda odpowiedź zapytania jest w języku polskim lub angielskim i dokładnie opisuje, co zostało wykonane.
- API posiada middleware, który zapisuje do pliku `log.txt` datę i czas (np. w formacie ISO), metodę HTTP oraz ścieżkę żądania.

\*Obraz 2 przedstawia bazę MySQL, w przypadku MongoDB `id` ma inną nazwę: `_id`

#### Założenia aplikacji

- Aplikacja komunikuje się z zapleczem poprzez API
- Do aplikacji został użyty Bootstrap, bez użycia własnego CSS-a.
- Kod aplikacji powinien być zapisany czytelnie, z zachowaniem zasad czystego formatowania
- Należy stosować znaczące nazwy zmiennych, metod i kontrolek
- Grafiki wykorzystywane w aplikacji pochodzą z archiwum dołączonego do zadania egzaminacyjnego

Podejmij próbę kompilacji i uruchomienia aplikacji. Wykonaj zrzut ekranowy stanu początkowego aplikacji i po pokazaniu kolejnego faktu i zdjęcia. Zrzuty ekranu zapisz w folderze webowa pod nazwami: `webowa1.png`, `webowa2.png`

Kod aplikacji przygotuj do nagrania na płytę. W podfolderze webowa należy utworzyć katalog frontend i backend (jeśli wymaga tego projekt). W katalogu `frontend/backend` powinno znaleźć się archiwum całego projektu o nazwie `frontend.zip/backend.zip` oraz plik (lub pliki) z kodem źródłowym modyfikowanym w czasie egzaminu.

### Część III. Testowanie aplikacji konsolowej

Wykonaj testy jednostkowe metody/funkcji obliczającej n-ty wyraz ciągu Fibonacciego, utworzonej w części I zadania (aplikacja konsolowa). Do napisania testów wykorzystaj dostępną na stanowisku egzaminacyjnym bibliotekę do testów jednostkowych. Testy jednostkowe powinny sprawdzać poprawność obliczania.

#### Założenia testów

- W każdej metodzie testującej należy użyć asercji
- Każdy przypadek testowy jest realizowany w osobnej metodzie.
- Nazwy metod testujących powinny jasno określać cel testu
- Wykonuje testy z tabeli 1

Tabela 1. Scenariusz testowania z przypadkami testowymi

| Nr testu | Cel testu<br>(Jaka sytuacja będzie testowana) | Dane wejściowe | Dane wyjściowe    |
|----------|-----------------------------------------------|----------------|-------------------|
| 1        | Sprawdzenie wyniku dla wyrazu równego 0       | 0              | 0                 |
| 2        | Sprawdzenie wyniku dla wyrazu równego 1       | 1              | 1                 |
| 3        | Sprawdzenie wyniku dla wyrazu równego 15      | 15             | 610               |
| 4        | Sprawdzenie wyniku dla wyrazu równego -1      | -1             | Wyjątek argumentu |

#### Uruchomienie testów i dokumentacja:

- Po napisaniu metod testujących uruchom wszystkie testy jednostkowe
- Wykonaj zrzut dokumentujący wyniki wykonania metod testujących
- Zrzuty ekranu powinny obejmować cały obszar ekranu, z widocznym paskiem zadań oraz środowiskiem programistycznym
- Zrzuty zapisz w folderze testy pod nazwami: *test1.png* itd.

Utwórz archiwum ZIP zawierające projekt testowy (jeśli to potrzebne) i zapisz w folderze testy. W folderze z numerem zdającego utwórz plik pakietu biurowego o nazwie **egzamin** i umieść w nim:

- nazwę systemu operacyjnego, na którym pracował zdający,
- nazwy środowisk programistycznych, z których zdający korzystał na egzaminie,
- nazwy języków programowania użytych podczas tworzenia aplikacji

Tabela 2. Selected fs (File System) methods

| Name                                                               | Description                                                                                                                                                                               |
|--------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| filehandle.appendFile(fileName, contentToAppend, format, callback) | Function used to add new content to an existing file. If the file does not exist, a new file is created. The callback function handles the result of the operation.                       |
| fsPromises.readdirSync(path)                                       | Function used to read the contents of a directory. Returns a list of files and folders from the specified path. The operation is synchronous, so the program waits until it is completed. |
| __dirname                                                          | Variable storing the full path of the directory where the current script file is located.                                                                                                 |

*UWAGA: Nagraj płytę z rezultatami pracy. W folderze z numerem zdającego powinny się znajdować podfoldery: konsola, webowa, testy i plik egzamin. W folderze konsola: cały projekt aplikacji konsolowej i zrzuty. W folderze webowa: cały projekt aplikacji webowej i zrzuty. W folderze testy: cały kod testów/archiwum aplikacji testującej, ewentualnie inne przygotowane pliki. Po nagraniu płyty sprawdź poprawność nagrania. Opisz płytę numerem zdającego i pozostaw na stanowisku, zapakowaną w pudełku wraz z arkuszem egzaminacyjnym.*

**Czas przeznaczony na wykonanie zadania wynosi 180 minut.**

**Ocenie będą podlegać 4 rezultaty:**

- implementacja, kompilacja, uruchomienie programu,
- aplikacja konsolowa,
- aplikacja webowa,
- testy aplikacji